

SOFTWARE ENGINEERING

UNIT - 4

Software Testing

Complete Syllabus Coverage

PYQ Integrated Topic-Wise | 4 Exam Papers

Simple Language | Easy to Learn & Write in Exam

Based on Pressman & Sommerville

HOW TO USE THESE NOTES

Structure of Each Topic

Every topic in this document follows the same flow:

1. **Theory First:** Concept explained in simple words with examples.
2. **Diagram:** Simple ASCII-style diagram you can draw in the exam.
3. **PYQ Questions:** The exact question as it was asked in previous exams.
4. **Short Answer:** 3-4 mark answer (3-5 lines). Write this if marks are low.
5. **Long Answer:** 7-10 mark answer (full detail). Write this for main questions.
6. **Memory Trick:** Quick tricks to remember lists and formulas.

Symbol	Meaning
* PYQ ALERT	This topic has been asked in previous year exams
! Exam Tip	Important exam writing advice
>> Memory Trick	Easy trick to remember the content
Orange Box	Exact PYQ question (do not change the wording)
Green Box	Answer section label

TOPIC 1: SOFTWARE TESTING FUNDAMENTALS What is testing? Why do we test?

Topic 1: Software Testing Fundamentals

1.1 What is Software Testing?

Think of it this way — when you buy a new phone, you check that calls work, apps open correctly, and the battery lasts. Similarly, software testing means checking that the software does what it is supposed to do — correctly, efficiently, and reliably.

Software Testing

Software testing is the process of executing a program with the intent of finding errors. It evaluates whether the software meets specified requirements and works correctly under all conditions. (Pressman)

1.2 Objectives of Testing

The main goal of testing is NOT to prove the software works — it is to FIND BUGS before the user does.

- To detect and remove errors from the software before delivery.
- To verify that the software satisfies all specified requirements.
- To validate that the software meets user expectations.
- To assess software quality and reliability before release.
- To ensure the software does not fail under unexpected conditions.
- To reduce risk and increase confidence in the product.
- To provide management with information about software quality.

>> **Memory Trick: Objectives = DETECT, VERIFY, VALIDATE, ASSESS, ENSURE. Remember: D-V-V-A-E**

1.3 Principles of Software Testing

These are the 7 golden rules of testing — always include these in long answers:

7. **Testing shows the presence of bugs, not their absence:** Even after testing, there may still be bugs hidden in the software.
8. **Exhaustive testing is impossible:** You cannot test every possible input. Instead, test the most important and risky cases.
9. **Early testing saves cost:** Bugs found early (in requirements or design) cost 10-100x less to fix than bugs found after release.
10. **Defect clustering:** Most bugs are found in a small number of modules. Focus more testing on complex or error-prone areas.
11. **Pesticide paradox:** Running the same tests repeatedly finds fewer bugs over time. Update and add new test cases regularly.
12. **Testing is context-dependent:** Testing a banking app requires more rigor than testing a simple student app.
13. **Absence of errors fallacy:** A bug-free software that does not meet user needs is still a failure.

>> Memory Trick: 7 Principles Memory Code: SB-EE-ET-DC-PP-CT-AE = 'Software Bugs Exist Even Though Done Carefully, People Pretend Carefully Testing Avoids Everything'

1.4 Testability

Testability means: how easy is it to test this software? High testability = fewer testing problems.

Testability Factor	Meaning	Example
Operability	Software works well enough to test	App doesn't crash during test setup
Observability	Output can be clearly seen	Test result shows pass/fail clearly
Controllability	Tester can control input/output	Can set any date in a booking system
Decomposability	Can test each module separately	Login module tested independently
Simplicity	Simple code is easier to test	Small functions = easy unit testing
Stability	Software doesn't change during testing	No code changes while testing a feature
Understandability	Tester understands the software	Good documentation helps testers

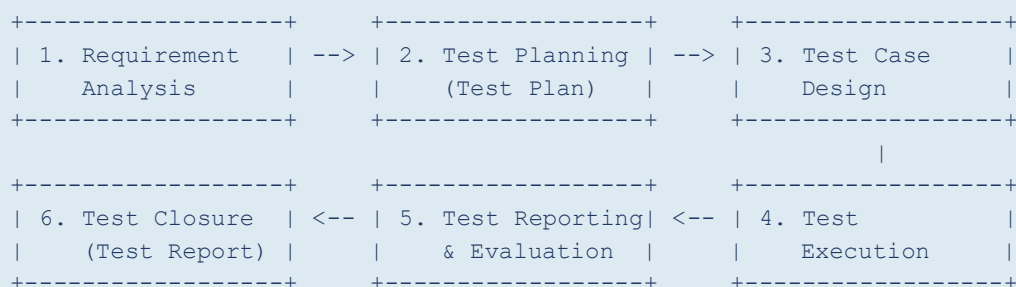
1.5 Test Case Design

A test case is a set of inputs, conditions, and expected outputs designed to test a specific feature. A good test case should:

- Have a clear objective and expected result.
- Be traceable to a requirement.
- Be repeatable — same inputs always give same results.
- Cover both valid and invalid inputs.

FIGURE: Software Testing Life Cycle (STLC)

SOFTWARE TESTING LIFE CYCLE



Key: --> = Next Phase <-- = Previous Phase

PYQ QUESTIONS — TOPIC 1: TESTING FUNDAMENTALS

Paper-1, Q4(a) | Paper-3, Q4(a) | 7-10 Marks [REPEATED]**Explain objectives and principles of software testing.**

*** PYQ ALERT: This question has been asked in multiple papers! If you learn this answer, you are safe for 7-10 marks.**

Short Answer (3-4 Marks):

The objective of software testing is to find and remove errors from the software before it is delivered to the user. The key principles include: testing shows presence of bugs (not absence), exhaustive testing is impossible, early testing saves cost, bugs cluster in small modules (defect clustering), the pesticide paradox says same tests find fewer bugs over time, testing depends on context, and absence of errors does not mean the software is acceptable.

Long Answer (7-10 Marks):**Definition of Software Testing:**

Software testing is the process of executing a program or system with the intent of finding errors. According to Pressman, testing evaluates software quality and improves it by identifying defects. It is a critical phase in the software development life cycle.

Objectives of Software Testing:

14. **Error Detection:** Find and correct defects before delivering software to the user.
15. **Verification:** Confirm the software meets its specified requirements correctly.
16. **Validation:** Ensure the software satisfies actual user needs and expectations.
17. **Quality Assessment:** Measure reliability, correctness, and performance of the software.
18. **Risk Reduction:** Reduce the probability of software failure in production.
19. **Information to Management:** Provide data about software quality for decision-making.

Principles of Software Testing (7 Principles by Pressman):

20. **Testing shows presence of bugs:** Testing can find bugs but cannot prove that zero bugs exist.
21. **Exhaustive testing impossible:** Testing all combinations of input is impractical. Testers use risk-based and priority-based approaches.
22. **Early testing:** Start testing as early as possible. Defects found in requirements phase cost 1x to fix; defects in production cost 100x.
23. **Defect clustering:** 80% of bugs are found in 20% of modules. Test complex and critical modules more thoroughly.
24. **Pesticide paradox:** Repeating the same tests loses effectiveness. Regularly update test cases to find new bugs.
25. **Testing is context-dependent:** A hospital patient management system needs more rigorous testing than a simple quiz app.
26. **Absence of errors fallacy:** A software with zero bugs but wrong functionality still fails user expectations.

Conclusion:

Software testing is not just finding bugs — it is a disciplined process guided by these principles to ensure quality, reliability, and user satisfaction.

TOPIC 2: WHITE BOX TESTING Structure-based testing — you can see the code

Topic 2: White Box Testing

2.1 What is White Box Testing?

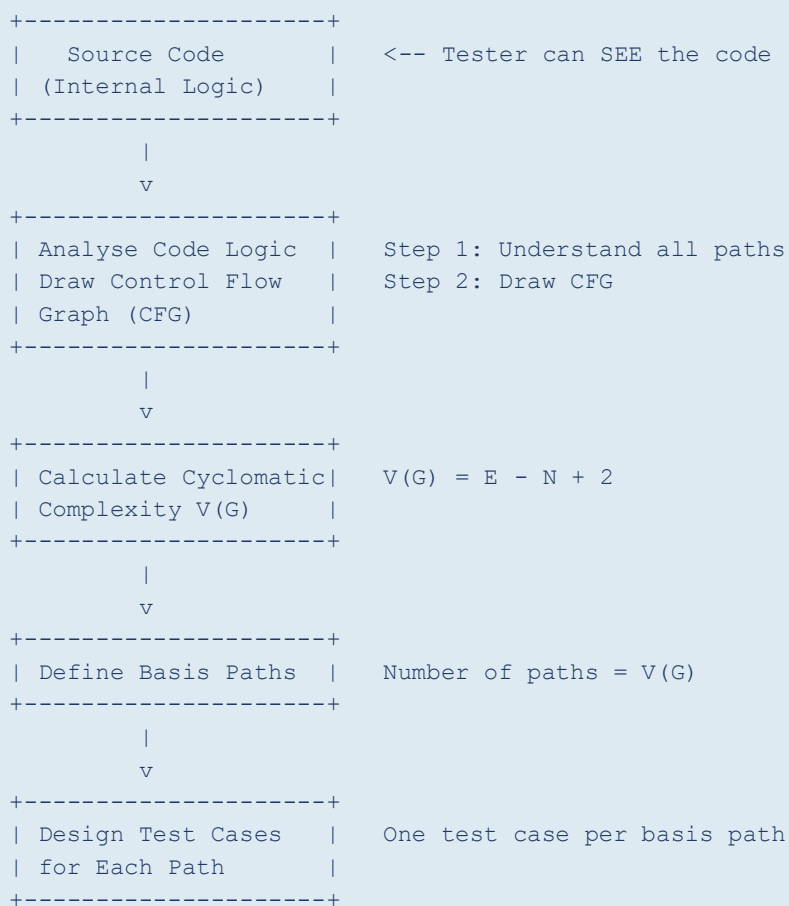
In White Box Testing, the tester can SEE the internal code of the software. It is like a glass box — you can look inside. The tester designs test cases based on the internal structure, logic, and code of the software.

White Box Testing

White Box Testing (also called Structural Testing or Glass Box Testing) is a testing technique where test cases are designed based on the internal structure of the program. The tester has full knowledge of the source code. (Pressman)

FIGURE: White Box Testing Flow

WHITE BOX TESTING FLOW



2.2 Basis Path Testing

Basis Path Testing is the most important White Box Testing technique. It was introduced by Tom McCabe. The idea is to find the minimum number of test cases needed to cover ALL paths in the program.

Steps of Basis Path Testing:

27. Draw the Control Flow Graph (CFG) from the program code.
28. Calculate Cyclomatic Complexity $V(G) = E - N + 2$.
29. Identify independent basis paths (number of paths = $V(G)$).
30. Design one test case for each basis path.
31. Execute test cases and verify results.

2.3 Control Flow Graph (CFG)

A Control Flow Graph (CFG) represents all possible paths that can be executed during a program's run. It shows how control flows from one statement to another.

CFG Elements:

Symbol	Name	Meaning
Circle (Node)	Process Node	A single statement or block of statements
Arrow (Edge)	Control Flow Edge	Direction of execution — from one node to next
Decision Diamond	Predicate Node	An IF or WHILE condition — creates 2 edges (True/False)
Entry Node	Start	Starting point of the program
Exit Node	End	Ending point of the program

FIGURE: Control Flow Graph (CFG) Example

CFG EXAMPLE (Simple If-Else Program)

CODE: ----- <pre> 1. Input X 2. IF X > 0 3. Print 'Positive' 4. ELSE 5. Print 'Non-Positive' 6. Print 'Done' 7. END </pre>	CFG: ---- <pre> (1) Entry (2) Decision [X > 0?] / \ (3) (4) <- Two branches True / False \ / (5) Merge (6) Exit </pre>
---	--

Edges (E) = 6 Nodes (N) = 6 $V(G) = 6 - 6 + 2 = 2$

2.4 Cyclomatic Complexity

Cyclomatic Complexity measures the complexity of a program by counting the number of linearly independent paths through the source code. It was introduced by Thomas McCabe in 1976.

CYCLOMATIC COMPLEXITY FORMULA:

$$V(G) = E - N + 2$$

Where:

E = Number of Edges in the CFG

N = Number of Nodes in the CFG

V(G) = Cyclomatic Complexity (Number of Independent Paths)

ALTERNATIVE FORMULAS:

V(G) = P + 1 (where P = number of predicate/decision nodes)

V(G) = Number of regions in the CFG (including outer region)

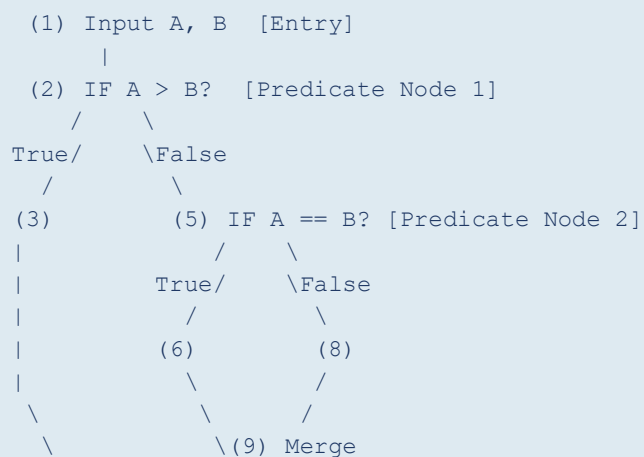
Step-by-Step Example: Calculating Cyclomatic Complexity

Consider this program:

1. Input A, B
2. IF A > B THEN
3. PRINT 'A is greater'
4. ELSE
5. IF A == B THEN
6. PRINT 'Equal'
7. ELSE
8. PRINT 'B is greater'
9. ENDIF
10. END

FIGURE: Cyclomatic Complexity Calculation

CFG for the above program:



```

      |
(10) END [Exit]

```

Count: $E = 9$, $N = 8$

$V(G) = E - N + 2 = 9 - 8 + 2 = 3$

OR: $P = 2$ predicate nodes, $V(G) = P + 1 = 2 + 1 = 3$

RESULT: 3 independent paths need 3 test cases

Interpretation of Cyclomatic Complexity:

V(G) Value	Risk Level	Recommendation
1 - 10	Low Risk	Well-structured code, easy to test
11 - 20	Moderate Risk	Moderately complex, review code
21 - 50	High Risk	Complex code, must refactor
Above 50	Very High Risk	Untestable code, immediate refactoring needed

! Exam Tip: In exams, always draw the CFG first, then count E and N, then apply $V(G) = E - N + 2$. Always mention the 3 formulas for V(G).

PYQ QUESTIONS — TOPIC 2: WHITE BOX TESTING

Paper-1, Q4(b) | Paper-4, Q4(a) | 7-10 Marks [REPEATED]

What is White Box Testing? Explain Basis Path Testing.

*** PYQ ALERT:** This question appears in Paper-1 and Paper-4. Learn CFG diagram and formula carefully.

Short Answer (3-4 Marks):

White Box Testing (also called Glass Box or Structural Testing) is a technique where test cases are designed based on the internal code structure. Basis Path Testing is a White Box technique that uses Cyclomatic Complexity $V(G) = E - N + 2$ to find the minimum number of test cases needed to cover all independent execution paths in the program.

Long Answer (7-10 Marks):

Definition:

White Box Testing is a testing approach where test cases are derived from the internal logic and structure of the program. The tester must have complete knowledge of the source code. It is also called Structural Testing, Clear Box Testing, or Glass Box Testing.

Basis Path Testing — Steps:

32. Draw the Control Flow Graph (CFG) from the source code.
33. **Calculate Cyclomatic Complexity:** $V(G) = E - N + 2$, where E = edges, N = nodes.
34. Identify all independent basis paths. The number of paths equals V(G).

35. Design one test case for each basis path so that every statement and decision is tested.
36. Execute tests and compare actual output with expected output.

Example:

For a program with $E=7$ edges and $N=6$ nodes: $V(G) = 7 - 6 + 2 = 3$. So 3 independent basis paths exist and minimum 3 test cases are needed.

Advantages of White Box Testing:

- Ensures all code paths, branches, and statements are tested.
- Helps detect hidden errors in internal logic.
- Effective for optimizing code quality.

Limitations:

- Cannot detect missing features (only tests existing code).
- Requires skilled testers who understand source code.

Paper-4, Q4(b) | 7-10 Marks

Define Cyclomatic Complexity. Explain its formula.

Short Answer (3-4 Marks):

Cyclomatic Complexity $V(G)$ measures the complexity of a program by counting independent paths in its Control Flow Graph. Formula: $V(G) = E - N + 2$, where E = number of edges, N = number of nodes. It tells the minimum number of test cases needed for full path coverage.

Long Answer (7-10 Marks):**Cyclomatic Complexity — Full Explanation:**

Cyclomatic Complexity was introduced by Thomas McCabe in 1976. It is a software metric that quantifies the number of linearly independent paths through a program's source code. It is used to measure the complexity of a program and helps determine the minimum number of test cases for complete path coverage.

Formulas:

- $V(G) = E - N + 2$ (E = Edges, N = Nodes in CFG)
- $V(G) = P + 1$ (P = Number of predicate/decision nodes)
- $V(G) = \text{Number of enclosed regions} + 1$ (including outer region)

Worked Example:

Program: IF $A > 0$ THEN print 'Positive' ELSE print 'Non-positive' END.

CFG has 4 nodes and 5 edges. $V(G) = 5 - 4 + 2 = 3$. Also, $P = 1$ decision node, so $V(G) = 1 + 1 = 2$. (Correct count depends on exact CFG drawn.)

This means 2 test cases are needed: one for $A > 0$ (True path) and one for $A \leq 0$ (False path).

TOPIC 3: BLACK BOX TESTING Specification-based testing — you cannot see the code

Topic 3: Black Box Testing

3.1 What is Black Box Testing?

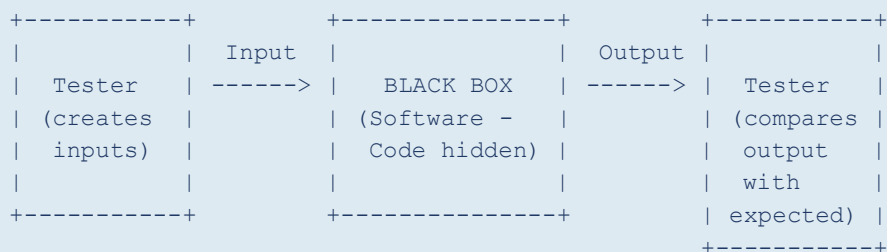
In Black Box Testing, the tester CANNOT see the internal code. It is like a black box — you only see what goes in (input) and what comes out (output). Test cases are designed based on REQUIREMENTS and SPECIFICATIONS only.

Black Box Testing

Black Box Testing (also called Behavioral Testing or Specification-Based Testing) is a testing method where test cases are designed from the software's specifications without any knowledge of the internal code structure. Only inputs and outputs are considered. (Pressman)

FIGURE: Black Box Testing Model

BLACK BOX TESTING MODEL



Tester only sees: INPUT and OUTPUT

Tester does NOT see: internal code, database, logic

3.2 Equivalence Class Partitioning (ECP)

Equivalence Class Partitioning divides all possible inputs into groups (partitions) where all values in a group are treated similarly by the software. Instead of testing every value, you test one value from each partition.

Types of Equivalence Classes:

- **Valid Equivalence Class:** Inputs that the system should accept and process correctly.
- **Invalid Equivalence Class:** Inputs that the system should reject or handle as errors.

Example — Age field (Valid range: 18 to 60):

Class	Type	Range	Sample Test Value	Expected Result
Class 1	Invalid	Age < 18 (e.g. 10)	10	Error: Too young
Class 2	Valid	18 <= Age <= 60 (e.g. 30)	30	Accept
Class 3	Invalid	Age > 60 (e.g. 70)	70	Error: Too old

Class 4	Invalid	Non-numeric (e.g. 'abc')	'abc'	Error: Invalid format
---------	---------	--------------------------	-------	-----------------------

! Exam Tip: ECP reduces the number of test cases while ensuring maximum coverage. Always mention: valid class, invalid class, and boundary values.

3.3 Boundary Value Analysis (BVA)

Boundary Value Analysis is based on the observation that most bugs occur at the BOUNDARIES of input ranges, not in the middle. BVA tests the values AT, JUST BELOW, and JUST ABOVE each boundary.

BOUNDARY VALUE ANALYSIS RULE:

For an input range [Min, Max]:

Test these values: Min-1, Min, Min+1, Typical Middle, Max-1, Max, Max+1

Example: Age range = 18 to 60

Test: 17 (below min), 18 (min), 19 (min+1),
35 (middle), 59 (max-1), 60 (max), 61 (above max)

BVA Example — Age field (Valid: 18 to 60):

Test Value	Category	Expected Result
17	Below Min Boundary	Reject — too young
18	Min Boundary (On)	Accept — valid
19	Just above Min (Min+1)	Accept — valid
35	Typical Middle Value	Accept — valid
59	Just below Max (Max-1)	Accept — valid
60	Max Boundary (On)	Accept — valid
61	Above Max Boundary	Reject — too old

>> Memory Trick: BVA = Test ON the boundary, JUST BELOW, and JUST ABOVE. Think: 'Edge cases are where bugs hide!'

PYQ QUESTIONS — TOPIC 3: BLACK BOX TESTING

Paper-1, Q4(c) | Paper-2, Q4(e) | Paper-3, Q4(b) | 7-10 Marks [REPEATED 3 TIMES]
Define Black Box Testing. Explain Equivalence Class Partitioning with example.

*** PYQ ALERT: Repeated in 3 papers! This is a MUST LEARN question. ECP example with table is essential.**

Short Answer (3-4 Marks):

Black Box Testing designs test cases purely from the software's specifications without knowledge of internal code. Equivalence Class Partitioning (ECP) divides inputs into groups where all values in a group behave the same way. For example, for age input valid range 18-60: valid class (18-60), invalid class below 18, invalid class above 60. Test one value from each class instead of testing every possible value.

Long Answer (7-10 Marks):**Black Box Testing — Definition:**

Black Box Testing is a specification-based testing technique where the tester has no knowledge of the internal code. Test cases are derived entirely from requirements and specifications. It is also called Behavioral Testing or Functional Testing. The tester provides inputs and verifies outputs against expected results.

Equivalence Class Partitioning — Full Explanation:

ECP divides the input domain into classes of data where each class represents a group of values that should all behave the same way. Instead of testing every value (exhaustive testing), we test one representative from each class. This reduces test cases while maintaining high defect detection coverage.

Rules for ECP:

37. For a continuous range of values, identify one valid and two invalid equivalence classes.
38. For a set of specific values, identify one valid class (a value in the set) and one invalid class (a value not in the set).
39. **For Boolean inputs, identify two classes:** true and false.

Detailed Example — Student Marks Entry (0 to 100):

Class No.	Class Type	Input Range	Test Value	Expected Output
EC1	Invalid (below)	Marks < 0	-5	Error: Invalid marks
EC2	Valid	0 <= Marks <= 100	75	Accept — marks recorded
EC3	Invalid (above)	Marks > 100	110	Error: Marks exceed limit
EC4	Invalid (type)	Non-numeric input	'abc'	Error: Enter numbers only

Advantage of ECP:

ECP reduces the number of test cases needed from potentially thousands (exhaustive) to just a few representative cases without losing significant coverage. It is time-efficient and highly practical.

Paper-1, Q4(c) | Paper-4, Q4(c) | Paper-3, Q4(b) | 7-10 Marks [REPEATED]
Explain Boundary Value Analysis with example.

Short Answer (3-4 Marks):

Boundary Value Analysis (BVA) tests input values at and near boundaries because bugs cluster at boundary conditions. For a valid range [Min, Max], BVA tests: Min-1, Min, Min+1, a middle value, Max-1, Max, Max+1. Example: age range 18-60, test values: 17, 18, 19, 35, 59, 60, 61.

Long Answer (7-10 Marks):

Boundary Value Analysis — Complete Explanation:

Boundary Value Analysis is a Black Box testing technique that focuses on testing at the boundary of input domains. Research shows that most programming errors occur at boundary conditions (off-by-one errors, incorrect loop conditions, etc.). BVA is often used alongside ECP.

BVA Testing Strategy:

For every valid range [Min, Max], test these 7 values: Min-1 (invalid), Min (valid), Min+1 (valid), Middle value (valid), Max-1 (valid), Max (valid), Max+1 (invalid).

Example — Password length (6 to 12 characters):

Test Value (Length)	Boundary Type	Expected Result
5	Below Min (Min-1)	Reject — too short
6	Min Boundary	Accept — valid
7	Min + 1	Accept — valid
9	Typical middle	Accept — valid
11	Max - 1	Accept — valid
12	Max Boundary	Accept — valid
13	Above Max (Max+1)	Reject — too long

BVA finds boundary-related bugs efficiently without testing every possible value in the range.

TOPIC 4: VERIFICATION AND VALIDATION Are we building the product right? Are we building the right product?

Topic 4: Software Verification and Validation (V&V)

4.1 Verification

Verification

Verification is the process of evaluating a software system to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase. It answers: 'Are we building the product RIGHT?' (IEEE)

Verification checks that the software is built correctly according to specifications. It involves internal reviews, walkthroughs, and inspections — without running the software. It catches bugs early.

Verification Activities:

- Requirements reviews and inspections.
- Design walkthroughs and reviews.
- Code reviews, static analysis, and peer review.
- Checking documents against standards.

4.2 Validation

Validation

Validation is the process of evaluating software during or at the end of development to determine whether it satisfies the specified requirements and user needs. It answers: 'Are we building the RIGHT product?' (IEEE)

Validation confirms that the final software product meets user needs and expectations. It involves actually running and testing the software — like unit testing, integration testing, system testing, and acceptance testing.

Validation Activities:

- Unit testing, integration testing, system testing.
- Acceptance testing with the client.
- Alpha and beta testing.
- Performance and stress testing.

4.3 Difference Between Verification and Validation

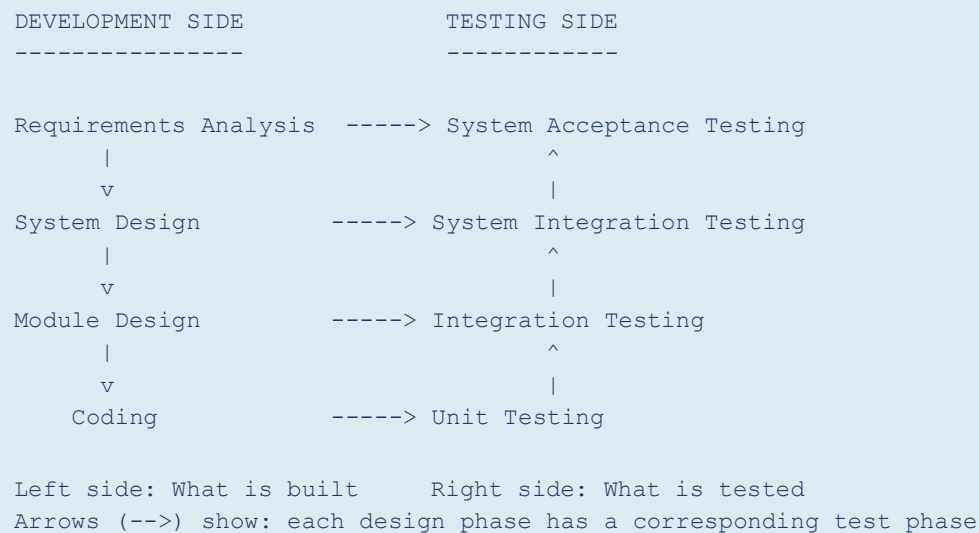
Point	Verification	Validation
Definition	Checks software is built correctly	Checks correct software is built
Question	Are we building it RIGHT?	Are we building the RIGHT thing?
Process	Static — reviews, walkthroughs	Dynamic — executing the software
Runs Software?	No (without execution)	Yes (with actual execution)
When?	During development phases	At end of development or after

Performed by	Internal team (developers, QA)	Testers and end users
Focus	Documents and code correctness	Functionality and user needs
Techniques	Code review, inspection, walkthrough	Unit test, integration test, UAT
Cost of errors	Cheaper to fix (early detection)	Expensive to fix (late detection)
Goal	Meet specifications	Meet user expectations

>> Memory Trick: Quick Memory: Verification = 'V for Valid spec' (internal). Validation = 'V for Valid to User' (external). Or: Verification=Review, Validation=Run.

FIGURE: V-Model Testing Diagram

V-MODEL TESTING DIAGRAM



PYQ QUESTIONS — TOPIC 4: VERIFICATION & VALIDATION

Paper-1, Q4(e) OR | Paper-3, Q4(d) | 7-10 Marks [REPEATED]

Explain Verification and Validation. Differentiate between them.

*** PYQ ALERT: This is the OR question in Paper-1 and also appears in Paper-3. The difference table is the most important part.**

Short Answer (3-4 Marks):

Verification checks that software is built correctly according to specifications (static — no execution, e.g., code reviews). Validation checks that the correct software was built according to user needs (dynamic — actual software execution, e.g., testing). Verification asks 'Are we building it right?' and Validation asks 'Are we building the right thing?'

Long Answer (7-10 Marks):**Verification — Detailed Explanation:**

Verification is a static quality control process that evaluates work products (documents, code, designs) without executing the software. It ensures the software correctly implements the requirements at each development phase. Verification activities include code inspection, design reviews, walkthroughs, and document checking. The goal is to detect defects early and cheaply.

Validation — Detailed Explanation:

Validation is a dynamic process that evaluates the final software product by actually running it. It confirms that the software meets the user's real-world needs and expectations. Validation activities include all forms of testing (unit, integration, system, acceptance testing). The goal is to ensure the delivered product does what the user actually needs.

V-Model: How V&V Works Together:

The V-Model shows that for every development phase on the left (requirements, design, coding), there is a corresponding testing phase on the right (acceptance testing, integration testing, unit testing). Verification and Validation work together throughout the lifecycle.

TOPIC 5: TYPES OF TESTING

Topic 5: Types of Software Testing

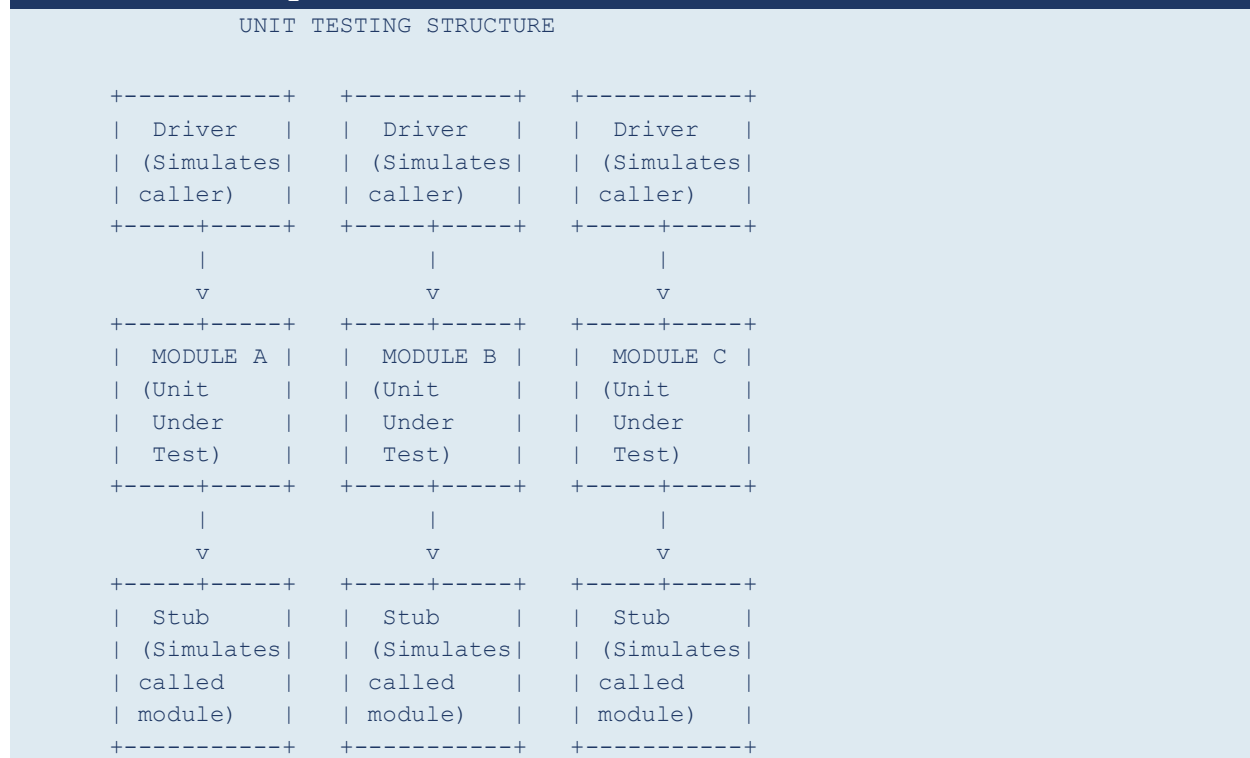
5.1 Unit Testing

Unit Testing

Unit Testing is a level of testing where individual components or modules of software are tested in isolation. A 'unit' is the smallest testable part — a function, method, or class. (Pressman)

Unit testing is performed by developers during the coding phase. The goal is to verify that each individual unit works correctly on its own before being combined with other units.

FIGURE: Unit Testing Structure with Drivers and Stubs



Key Terms:

- **Driver:** A module that calls the unit under test (simulates the calling module).
- **Stub:** A dummy module that simulates a module called by the unit under test.

5.2 Integration Testing

Integration Testing

Integration Testing is the phase of software testing where individual units are combined and tested as a group to evaluate the interfaces between them and the interactions within the combined system. (Sommerville)

After unit testing, modules are integrated and tested together. The goal is to find interface defects — errors in how modules communicate with each other.

Types of Integration Testing:

40. **Big Bang Integration:** All modules are integrated at once and tested together. Simple but hard to isolate faults.
41. **Top-Down Integration:** Testing starts from the top-level module and works downward using stubs for lower modules.
42. **Bottom-Up Integration:** Testing starts from the lowest-level modules and works upward using drivers for higher modules.
43. **Sandwich/Hybrid Integration:** Combines top-down and bottom-up approaches simultaneously.

FIGURE: Integration Testing Types

INTEGRATION TESTING TYPES

BIG BANG: ----- Module A \ Module B } ----> Module C / (All at once)	TOP-DOWN: ----- [TOP MODULE] / \ M1 M2 M3 M1a M2a M3a (Stubs used below)	BOTTOM-UP: ----- Module A Module B } --> Drivers Module C (Low level first)
--	--	---

Big Bang: Fast but hard to find bugs
 Top-Down: Tests main logic first, needs stubs
 Bottom-Up: Tests components first, needs drivers

5.3 Regression Testing

Regression Testing

Regression Testing is the process of re-testing software after changes (bug fixes, new features, or enhancements) to ensure that previously working functions have not been broken by the changes. (Pressman)

When a developer fixes a bug or adds a new feature, they may accidentally break something that was working before. Regression testing re-runs existing test cases to ensure nothing is broken.

Why is Regression Testing Important?

- Prevents previously fixed bugs from re-appearing (called regressions).
- Ensures new code does not break existing functionality.
- Essential in Agile and continuous development environments.
- Builds confidence in software stability after every change.

5.4 Alpha Testing

Alpha Testing

Alpha Testing is a type of acceptance testing performed at the developer's site. It is done by internal employees or a small group of trusted users, not the actual end users. The development team observes and records defects. (Pressman)

Alpha testing is the first pass of user-level testing. It happens before the product is released to the public. The software is tested in a controlled environment by a selected group.

5.5 Beta Testing

Beta Testing

Beta Testing is a form of acceptance testing performed at the customer's site, without the development team being present. Actual end users test the software in a real-world environment and report defects. (Pressman)

Beta testing is the final pre-release testing phase. Real users test the product at their own location without developer supervision. This gives realistic feedback from real usage conditions.

5.6 Comparison: All Testing Types

Testing Type	Who Tests	Where	Software State	Purpose
Unit Testing	Developers	Development environment	Individual module	Test single units
Integration Testing	Dev/Test Team	Development environment	Combined modules	Test interfaces
Regression Testing	QA Team	Test environment	Modified software	Ensure no breakage
Alpha Testing	Internal employees	Developer's site	Near-complete	Pre-release testing
Beta Testing	Actual end users	User's environment	Release candidate	Real-world testing
System Testing	Test team	Test environment	Complete system	Full system check

>> **Memory Trick: Testing Order Memory: Unit -> Integration -> System -> Regression -> Alpha -> Beta -> Release. Mnemonic: 'Uncle Integrates Systems Rapidly. Alice Brings Release.'**

PYQ QUESTIONS — TOPIC 5: TYPES OF TESTING

Paper-2, Q4(a) | 7-10 Marks

Explain Unit Testing with example.

Short Answer (3-4 Marks):

Unit Testing tests the smallest individual unit of software (a function, method, or class) in isolation. Example: testing a function 'calculateTax(salary)' separately with different salary inputs before integrating it into the payroll system. Drivers simulate calling modules and stubs simulate called modules.

Long Answer (7-10 Marks):**Unit Testing — Complete Explanation:**

Unit Testing is the first level of formal testing in the SDLC, performed during the coding phase. Each unit (smallest testable part of the software) is tested independently to verify correct behavior. Unit tests are typically written and run by developers using testing frameworks.

What is tested in Unit Testing:

- **Interface:** Do parameters pass correctly to and from the unit?
- **Local data structures:** Are variables initialized correctly?
- **Boundary conditions:** Does the unit handle edge cases correctly?
- **Error handling:** Does the unit handle invalid inputs gracefully?
- **Logic paths:** Are all if-else branches executed correctly?

Practical Example:

Consider a function: calculateGrade(marks). Test cases:

- **Input:** marks = 95 → Expected: 'A+'
- **Input:** marks = 75 → Expected: 'B'
- **Input:** marks = -5 → Expected: Error (invalid input)
- **Input:** marks = 101 → Expected: Error (out of range)

Each test case is run independently to verify the function works correctly for all scenarios.

Paper-1, Q4(d) | 7-10 Marks

Explain Integration Testing. Write its types.

Short Answer (3-4 Marks):

Integration Testing combines individual modules and tests them as a group to verify interface correctness. Types: Big Bang (all at once), Top-Down (from top module using stubs), Bottom-Up (from lower modules using drivers), and Sandwich/Hybrid (combination of both).

Long Answer (7-10 Marks):**Integration Testing — Full Explanation:**

Integration Testing is performed after unit testing, when individual modules are combined to form subsystems or the complete system. The objective is to detect interface faults — errors that occur when modules interact with each other. Even if each module passes unit testing, integration errors can occur due to incorrect data transfer, wrong function calls, or timing issues.

Types of Integration Testing:

44. **Big Bang Integration Testing:** All modules are combined and tested simultaneously.
Advantage: Simple and fast setup. Disadvantage: Very hard to isolate which module caused a bug. Used for small systems.
45. **Top-Down Integration Testing:** Integration starts with the top-level module (main control module). Lower-level modules are replaced by stubs (dummy modules). Stubs are progressively replaced with actual modules. Advantage: Tests main logic early. Disadvantage: Requires many stubs.

46. **Bottom-Up Integration Testing:** Integration starts with the lowest-level modules (leaves of the module hierarchy). Higher-level modules are simulated using drivers. Drivers are progressively replaced with actual higher modules. Advantage: No stubs needed. Disadvantage: High-level logic tested late.
47. **Sandwich/Hybrid Integration:** Combines both Top-Down and Bottom-Up. Middle layer is tested using both stubs (below) and drivers (above). Advantage: More efficient. Used in large complex systems.

Paper-2, Q4(c) | 7-10 Marks**What is Regression Testing? Explain its importance.****Short Answer (3-4 Marks):**

Regression Testing re-tests software after changes (bug fixes or new features) to ensure previously working functions still work correctly. It prevents regressions — situations where fixing one bug breaks another feature. It is critical in Agile and continuous development.

Long Answer (7-10 Marks):**Regression Testing — Complete Explanation:**

Regression Testing is re-execution of previously run tests after a change to the software. The purpose is to verify that existing functionality has not been broken by changes such as bug fixes, new feature additions, or performance improvements. It is also called 'confirmation testing' or 're-testing'.

Why is Regression Testing Important:

48. **Prevents side effects:** A bug fix in one module may introduce new bugs in other modules. Regression testing catches these.
49. **Maintains software quality:** Ensures product quality is maintained across all versions and releases.
50. **Supports agile development:** In agile, software is updated frequently. Regression testing ensures each sprint doesn't break old features.
51. **Automated regression:** Most regression tests are automated (using tools like Selenium, QTP) to run quickly after every change.
52. **Increases confidence:** Teams can make changes confidently knowing regression tests will catch unexpected breakage.

When to Run Regression Testing:

- After every bug fix is committed.
- After adding new features.
- After code refactoring.
- Before every major release.

Paper-2, Q4(d) | 3-4 Marks**Define Alpha Testing and Beta Testing.****Short Answer (3-4 Marks):**

Alpha Testing is conducted at the developer's site by internal employees before release. It is the first user-level test in a controlled environment with developer observation. Beta Testing is conducted at the customer's site by actual end users without developer presence. It tests the software under real-world conditions. Alpha comes before Beta. Together they form User Acceptance Testing (UAT).

Feature	Alpha Testing	Beta Testing
Who	Internal employees / trusted testers	Actual end users / customers
Where	Developer's site	Customer's environment
Developer Present?	Yes — observes and records defects	No — users self-report issues
Controlled?	Highly controlled environment	Real-world uncontrolled
Stage	Before Beta testing	After Alpha, before release
Feedback	Immediate, detailed feedback	User experience feedback
Example	Microsoft employees test Windows	Public beta of Chrome browser

TOPIC 6: TEST DOCUMENTATION Test Plan, Test Case, Test Report

Topic 6: Test Documentation

Test documentation provides a formal record of testing activities. Good documentation ensures reproducibility, traceability, and accountability in the testing process.

6.1 Test Plan

Test Plan

A Test Plan is a formal document that describes the scope, approach, resources, and schedule for testing activities. It outlines what will be tested, how it will be tested, who will test it, and when testing will occur. (IEEE 829)

Contents of a Test Plan:

Section	Content
1. Test Plan Identifier	Unique ID for the test plan
2. Introduction	Overview of the software and testing objectives
3. Test Items	List of features and modules to be tested
4. Features to be Tested	Specific functionalities to test
5. Features Not to be Tested	Out-of-scope items
6. Testing Approach	White box, black box, or combination
7. Test Environment	Hardware, software, tools required
8. Schedule	Timeline for each testing phase
9. Resources	Team members, their roles, and responsibilities
10. Risks and Contingencies	Potential problems and backup plans
11. Approval	Signatures of stakeholders

6.2 Test Case

Test Case

A Test Case is a document specifying a set of inputs, preconditions, execution conditions, expected outputs, and postconditions designed to test one particular aspect of a software program. (IEEE 829)

Sample Test Case Format:

Field	Description	Example
Test Case ID	Unique identifier	TC_LOGIN_001
Test Case Name	Short description	Valid Login Test
Module	Which module is tested	Login Module
Preconditions	Required setup before test	User registered, app is open
Test Data	Specific inputs to use	Username: admin, Password: admin123

Test Steps	Steps to execute	1. Open app 2. Enter username 3. Enter password 4. Click Login
Expected Result	What should happen	Dashboard page opens successfully
Actual Result	What actually happened	Dashboard opened (filled after execution)
Status	Pass or Fail	Pass
Tested By	Tester's name	Rahul Sharma
Date	Date of execution	01-May-2026

6.3 Test Report

Test Report

A Test Report (or Test Summary Report) is a document that summarizes the testing activities, results, defects found, and the overall quality assessment of the software. It is prepared after testing is complete. (IEEE 829)

Contents of a Test Report:

Section	Content
Project Name & Version	Software being tested
Test Period	Start and end date of testing
Total Test Cases	Total number of test cases executed
Test Cases Passed	Number of test cases passed
Test Cases Failed	Number of test cases failed
Defects Found	List of bugs with severity
Defects Fixed	Bugs resolved during testing
Test Coverage	Percentage of features tested
Overall Assessment	Pass/Fail recommendation for release
Prepared By	Test team names and date

TOPIC 7: SOFTWARE TESTING TOOLS QTP, Rational Robot, WinRunner, LoadRunner

Topic 7: Software Testing Tools

Testing tools automate the testing process, making it faster, more accurate, and repeatable. They are essential for regression testing and performance testing in large projects.

7.1 QTP (Quick Test Professional)

QTP

Quick Test Professional (QTP), now known as HP UFT (Unified Functional Testing), is an automated functional testing tool by HP (Hewlett Packard). It tests web applications, Windows apps, and web services using VBScript language.

Feature	Details
Full Name	Quick Test Professional (now HP UFT — Unified Functional Testing)
Developer	HP (Hewlett Packard)
Language	VBScript
Testing Type	Functional and Regression Testing
Applications	Web, Windows Desktop, SAP, Java applications
Key Feature 1	Record and Playback: Records user actions and replays them
Key Feature 2	Object Repository: Stores all UI elements for reuse
Key Feature 3	Data-driven testing with Excel integration
Key Feature 4	Generates detailed HTML test reports automatically
Advantage	Easy to learn, widely used in industry, good reporting

7.2 Rational Robot

Rational Robot

Rational Robot is an automated testing tool developed by IBM Rational. It supports functional, regression, and performance testing of Windows applications, web applications, and Java applications using SQABasic scripting language.

Feature	Details
Developer	IBM Rational
Language	SQABasic
Testing Type	Functional, Regression, Performance
Applications	Windows, Web, Java, SAP applications
Key Feature 1	GUI-based record and playback
Key Feature 2	Integration with IBM Rational test management tools
Key Feature 3	Supports object-level testing and verification

Key Feature 4	Detailed log files and comparison reports
Advantage	Excellent for IBM/Rational development environments
Limitation	Expensive; mostly replaced by newer tools

7.3 WinRunner

WinRunner	WinRunner is an automated functional testing tool developed by Mercury Interactive (later acquired by HP). It tests Windows-based GUI applications by recording user actions and replaying them. It uses TSL (Test Script Language).
------------------	--

Feature	Details
Developer	Mercury Interactive (now HP)
Language	TSL (Test Script Language)
Testing Type	Functional GUI Testing, Regression Testing
Applications	Windows desktop GUI applications
Key Feature 1	Record and Playback of GUI actions
Key Feature 2	Data-driven testing using tables
Key Feature 3	Bitmap comparison for visual verification
Key Feature 4	GUI Map to manage application objects
Note	WinRunner is now discontinued and replaced by HP UFT (QTP)

7.4 LoadRunner

LoadRunner	LoadRunner is an industry-leading performance testing tool developed by HP. It tests the behavior and performance of a system under load by simulating thousands of concurrent virtual users to identify performance bottlenecks.
-------------------	---

Feature	Details
Developer	HP (Hewlett Packard)
Testing Type	Performance, Load, Stress, Endurance Testing
Key Feature 1	Virtual User Generator (VuGen): Records user actions as scripts
Key Feature 2	Controller: Manages and controls load test scenarios
Key Feature 3	Load Generators: Simulate thousands of concurrent users
Key Feature 4	Analysis Module: Provides detailed performance graphs and reports
Simulates	Up to thousands of concurrent virtual users from one machine
Key Metrics	Response time, throughput, server CPU/memory utilization
Use Case	Banks, e-commerce sites testing peak load (e.g., Black Friday traffic)
Advantage	Industry standard; comprehensive analytics and reporting

7.5 Comparison of Testing Tools

Tool	Developer	Testing Type	Language	Best For
QTP / UFT	HP	Functional, Regression	VBScript	Web and desktop automation
Rational Robot	IBM	Functional, Performance	SQABasic	IBM Rational environments
WinRunner	Mercury/HP	Functional GUI	TSL	Windows GUI (now obsolete)
LoadRunner	HP	Performance, Load, Stress	C, VBScript	Load and performance testing

! Exam Tip: In exams, always list: Full Name, Developer, Testing Type, and 2-3 Key Features for each tool. This gets full marks in 7-10 mark questions.

PYQ QUESTIONS — TOPIC 7: TESTING TOOLS

Paper-4, Q4(e) OR | 7-10 Marks

Explain Testing Tools such as QTP and LoadRunner.

Short Answer (3-4 Marks):

QTP (Quick Test Professional) by HP is an automated functional testing tool that uses VBScript and supports record-and-playback testing of web and desktop applications. LoadRunner by HP is a performance testing tool that simulates thousands of virtual users to test system behavior under load conditions, identifying performance bottlenecks using its VuGen, Controller, and Analysis modules.

Long Answer (7-10 Marks):

QTP (Quick Test Professional) — Detailed Explanation:

QTP, now called HP UFT (Unified Functional Testing), is an automated functional and regression testing tool. It allows testers to record their interactions with an application and play them back automatically.

QTP Key Features:

- **Record and Playback:** Automatically records user actions and replays them.
- **Object Repository:** Stores all UI elements centrally for easy reuse.
- **Data-Driven Testing:** Uses Excel sheets to run tests with multiple data sets.
- **Multi-application support:** Tests web, Windows, Java, SAP applications.
- Automatic report generation in HTML format.

LoadRunner — Detailed Explanation:

LoadRunner is HP's performance testing solution designed to test how an application behaves when many users access it simultaneously. It simulates the load of thousands of concurrent users to find performance bottlenecks before production release.

LoadRunner Key Components:

- **Virtual User Generator (VuGen):** Records and creates load test scripts.
- **Controller:** Defines and manages the load test scenario (number of users, ramp-up time).
- **Load Generators:** Machines that simulate virtual users.
- **Analysis Module:** Generates performance graphs for response time, throughput, and server metrics.

LoadRunner Example Use Case:

An online banking portal expects 10,000 users at peak times. LoadRunner simulates 10,000 virtual users logging in simultaneously to identify if the server can handle the load without performance degradation.

MOST IMPORTANT QUESTIONS FOR EXAM Based on PYQ Frequency Analysis

VERY HIGH PRIORITY (Repeated in 3+ papers — MUST LEARN)

#	Question	Papers	Marks
1	Objectives and Principles of Software Testing	P1, P3	7-10
2	Black Box Testing + Equivalence Class Partitioning with example	P1, P2, P3	7-10
3	White Box Testing + Basis Path Testing	P1, P4	7-10
4	Cyclomatic Complexity + Formula + Example	P2, P4	7-10
5	Verification and Validation + Difference Table	P1, P3	7-10

HIGH PRIORITY (Repeated in 2 papers — Should Learn)

#	Question	Papers	Marks
1	Integration Testing + Types (Big Bang, Top-Down, Bottom-Up)	P1, P4	7-10
2	Boundary Value Analysis with example	P1, P4	7-10
3	Control Flow Graph (CFG) + Importance	P3, P4	7-10
4	Unit Testing with example	P2, P4	7-10
5	Alpha Testing and Beta Testing + Difference	P2	3-4

MEDIUM PRIORITY (Single paper — Good to Learn)

#	Question	Papers	Marks
1	Regression Testing + Importance	P2	7-10
2	Testing Tools: QTP and LoadRunner	P4	7-10
3	Test Plan, Test Case, Test Report	P3	3-4
4	Testability factors	P3	3-4

QUICK REVISION SECTION Key Definitions, Formulas, Diagrams, Memory Tricks

Key Definitions to Remember

Term	One-Line Definition
Software Testing	Executing program with intent of finding errors (Pressman)
White Box Testing	Test cases based on internal code structure (glass box testing)
Black Box Testing	Test cases based on specifications without knowledge of code
Basis Path Testing	McCabe's technique to find minimum test cases covering all paths
Control Flow Graph	Graph showing all possible execution paths in a program
Cyclomatic Complexity	$V(G) = E - N + 2$; measures number of independent paths
ECP	Equivalence Class Partitioning — divides inputs into valid/invalid groups
BVA	Boundary Value Analysis — tests at and near boundary values of input ranges
Verification	Process of checking software is built correctly (static — no execution)
Validation	Process of checking correct software was built (dynamic — execution)
Unit Testing	Testing smallest individual component in isolation
Integration Testing	Testing combined modules for interface correctness
Regression Testing	Re-testing after changes to ensure no existing functionality is broken
Alpha Testing	Internal user testing at developer's site before release
Beta Testing	External user testing at customer's site before final release
QTP	HP's automated functional testing tool using VBScript
LoadRunner	HP's performance tool simulating thousands of virtual users

Important Formulas

CYCLOMATIC COMPLEXITY:

$$V(G) = E - N + 2 \quad (\text{Primary formula})$$

$$V(G) = P + 1 \quad (P = \text{number of predicate/decision nodes})$$

$$V(G) = \text{Regions} + 1 \quad (\text{count enclosed regions in CFG})$$

RISK LEVELS:

$$V(G) = 1-10 : \text{Low risk (good code)}$$

$$V(G) = 11-20 : \text{Moderate risk}$$

$$V(G) = 21-50 : \text{High risk (complex code)}$$

$$V(G) > 50 : \text{Very high risk (untestable)}$$

BVA TEST VALUES FOR RANGE [Min, Max]:

Test: Min-1, Min, Min+1, Middle, Max-1, Max, Max+1 (7 values)

Diagram Revision List

- Software Testing Life Cycle (STLC) — 6-phase cycle diagram.
- White Box Testing Flow — Source Code → CFG → Cyclomatic Complexity → Basis Paths → Test Cases.
- Control Flow Graph (CFG) — Nodes, Edges, Predicate nodes, Entry/Exit.
- Cyclomatic Complexity Example — Draw CFG, count E and N, calculate $V(G)$.
- Black Box Testing Model — Input → Black Box → Output comparison.
- Integration Testing Types — Big Bang, Top-Down (with stubs), Bottom-Up (with drivers).
- **V-Model Testing Diagram** — Left: Development phases, Right: Testing phases.
- Unit Testing Structure — Module + Driver (above) + Stub (below).

Memory Tricks Summary

>> Memory Trick: 7 Testing Principles: SB-EE-ET-DC-PP-CT-AE (Shows Bugs, Exhaustive impossible, Early testing, Defect cluster, Pesticide paradox, Context, Absence fallacy)

>> Memory Trick: McCall Quality Factors: C-R-E-I-U (Operation) + M-F-T (Revision) + P-R-I (Transition) = CREIUMFTPRI

>> Memory Trick: Testing Order: Uncle Integrates Systems Rapidly. Alice Brings Release. (Unit, Integration, System, Regression, Alpha, Beta, Release)

>> Memory Trick: V&V: Verification = Review (are we building it RIGHT?). Validation = Run (are we building the RIGHT thing?)

>> Memory Trick: BVA = Test 7 values per range: Min-1, Min, Min+1, Middle, Max-1, Max, Max+1

>> Memory Trick: Cyclomatic Complexity Formula: $V(G) = E - N + 2$. Remember: 'Every Network has 2 extras' (E minus N plus 2)

Exam Writing Tips

! Exam Tip: Always start your answer with a definition. Use numbered/bulleted points. Include at least one example. Draw a diagram if possible. End with a conclusion line.

! Exam Tip: For Cyclomatic Complexity questions: Always draw the CFG first, label all nodes and edges, count E and N, then calculate $V(G)$. This shows complete understanding.

! Exam Tip: For Black Box Testing questions: Always mention BOTH ECP and BVA techniques. Give a numerical example table for at least one technique.

! Exam Tip: For V&V questions: The Difference Table between Verification and Validation is the most important part. Draw it clearly with at least 5-6 points of difference.

END OF UNIT-4 — SOFTWARE TESTING MASTER NOTES

Best of Luck for Your Exams!